

9.5 Mecanum Wheel Chassis Speed Control

Speed control is achieved by adjusting the linear and angular velocity parameter.

1. Program Logic

Based on the robot's movement characteristics, control the active wheels to achieve forward, backward, and turning movements.

In the program, subscribe to the '/controller/cmd_vel' movement control topic to obtain the set linear and angular velocities. Then, analyze and calculate based on these velocities to determine the car's movement speed.

The source code for this program can be found at:

/home/ubuntu/ros2_ws/src/driver/controller/controller/odom_publisher_node.py

```
70 class Controller(Node):
71
72     def __init__(self, name):
73         rclpy.init()
74         super().__init__(name)
75
76         self.x = 0.0
77         self.y = 0.0
78         self.linear_x = 0.0
79         self.linear_y = 0.0
80         self.angular_z = 0.0
81         self.pose_yaw = 0
82         self.last_time = None
83         self.current_time = None
84         signal.signal(signal.SIGINT, self.shutdown)
85
86         self.ackermann = ackermann.AckermannChassis(wheelbase=0.145, track_width=0.133, wheel_diameter=0.067)
87         self.mecanum = mecanum.MecanumChassis(wheelbase=0.1368, track_width=0.1446, wheel_diameter=0.065)
88
89         # Declare parameters
90         self.declare_parameter('pub_odom_topic', True)
91         self.declare_parameter('base_frame_id', 'base_footprint')
92         self.declare_parameter('odom_frame_id', 'odom')
93         self.declare_parameter('linear_correction_factor', 1.00)
94         self.declare_parameter('linear_correction_factor_tank', 0.52)
95         self.declare_parameter('angular_correction_factor', 1.00)
96         self.declare_parameter('machine_type', os.environ['MACHINE_TYPE'])
97
98         self.pub_odom_topic = self.get_parameter('pub_odom_topic').value
99         self.base_frame_id = self.get_parameter('base_frame_id').value
100        self.odom_frame_id = self.get_parameter('odom_frame_id').value
```

2. Disable APP Service and Initiate Speed Control

Note: The input command is case-sensitive, and keywords can be completed using the Tab key.

- 1) Start the robot, and connect it to the robot system desktop using VNC.



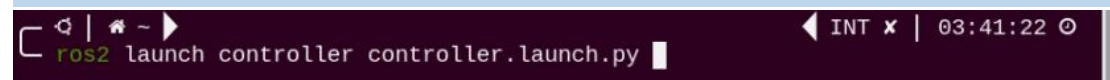
- 2) Click-on  to open the command-line terminal.
- 3) Enter the command and hit Enter to disable the app auto-start service.

```
~/stop_ros.sh
```



- 4) Enter the command to enable motion control service.

```
ros2 launch controller controller.launch.py
```



2.1 Modify Linear Velocity

- 1) Open a new ROS2 command-line terminal, then enter the following command to enable the linear velocity control.

```
ros2 topic pub /controller/cmd_vel geometry_msgs/Twist "linear:
```

```
  x: 0.0
```

```
  y: 0.0
```

```
  z: 0.0
```

```
angular:
```

```
  x: 0.0
```

```
  y: 0.0
```

```
  z: 0.0"
```

```
> ros2 topic pub /controller/cmd_vel geometry_msgs/Twist "linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0),
publishing #2: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0),
```

In this context, '**linear**' represents the set linear velocity, considering the robot's viewpoint where the X-axis points forward without any influence from the Y or Z directions.

On the other hand, '**angular**' pertains to the set angular velocity. A positive

Z-value induces a left turn in the robot, while a negative Z-value causes the robot to turn right. This configuration has no impact on the X and Y directions.

Note:

1. In this scenario, the linear velocity (x) is measured in meters per second, and it is advisable to maintain it within the range of "-0.6 to 0.6".
 2. The angular velocity (z) denotes the turning speed and is determined by the formulas $V=\omega R$ (linear velocity equals angular velocity times radius) and $\tan\Phi A=D/R$ (where $z=\omega$, $D=0.213$, and ΦA represents the turning angle). The angle ΦA should be within the range of 0 to 36 degrees.
-

"angular" sets the angular velocity of the robot. A positive value for "Z" will make the robot turn left, while a negative value for "Z" will make it turn right. There is no motion in the X and Y directions.

- 2) Use the arrow keys to navigate and modify the relevant parameters. For example, to make the robot move forward, adjust the linear velocity (X) to 0.1, and then press Enter to execute the action.

```
* ros2 topic pub /controller/cmd_vel geometry_msgs/Twist "linear:
x: 0.1
y: 0.0
z: 0.0
angular:
x: 0.0
y: 0.0
z: 0.0"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.1, y=0.0, z=0.0)
```

- 3) To bring the robot car to a stop, open a new terminal and set the linear velocity to '0.0'.

```
> ros2 topic pub /controller/cmd_vel geometry_msgs/Twist "linear:
x: 0.0
y: 0.0
z: 0.0
angular:
x: 0.0
y: 0.0
z: 0.0"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0),
```

- 4) If you need to terminate this program, use short-cut 'Ctrl+C'.
-

Note: To bring the robot car to a stop, please create a new terminal and adjust the linear velocity. Using the 'Ctrl+C' shortcut alone may not effectively halt the robot car.

2.2 Modify Angular Velocity

1) Open a new ROS2 command-line terminal, then enter the following command to enable the angular velocity control.

```
ros2 topic pub /controller/cmd_vel geometry_msgs/Twist "linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0"
> ros2 topic pub /controller/cmd_vel geometry_msgs/Twist "linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0),
publishing #2: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0),
```

"angular" sets the angular velocity of the robot. A positive value for "Z" will make the robot turn left, while a negative value for "Z" will make it turn right. There is no motion in the X and Y directions.

Note:

1. In this scenario, the linear velocity (x) is measured in meters per second, and it is advisable to maintain it within the range of "-0.6 to 0.6".
2. "Z" represents the angular velocity for turning, which is the yaw angular velocity of the robot. It is recommended to set the value between "-2" and "2". A value of "0" means there is no angular velocity, a positive value means counterclockwise rotation, and a negative value means clockwise rotation. The absolute value of the angular velocity represents the speed of rotation, with larger values indicating faster rotation.

2) Use the arrow keys to navigate and modify the relevant parameters. For example, to make the robot turn left, adjust the angular velocity (Z) to 1.0, and

then press Enter to execute the action.

```

ros2 topic pub /controller/cmd_vel geometry_msgs/Twist "linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 1.0"

```

3) To bring the robot car to a stop, open a new terminal and set the angular velocity to '0.0'.

```

> ros2 topic pub /controller/cmd_vel geometry_msgs/Twist "linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0),

```

4) If you want to exit the game, press "Ctrl+C" in the terminal interface.

Note: Create a new terminal to stop the robot, and then close the node to shut down properly. If you directly press Ctrl+C to close the terminal, the robot may not stop.

After experiencing the game, you can enable the app service through commands or by restarting the robot. If the app is not enabled, the related app functions will not work. If the robot is restarted, the app will be automatically enabled.



Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

Note: please enter the command in the system path, not in the Docker container.

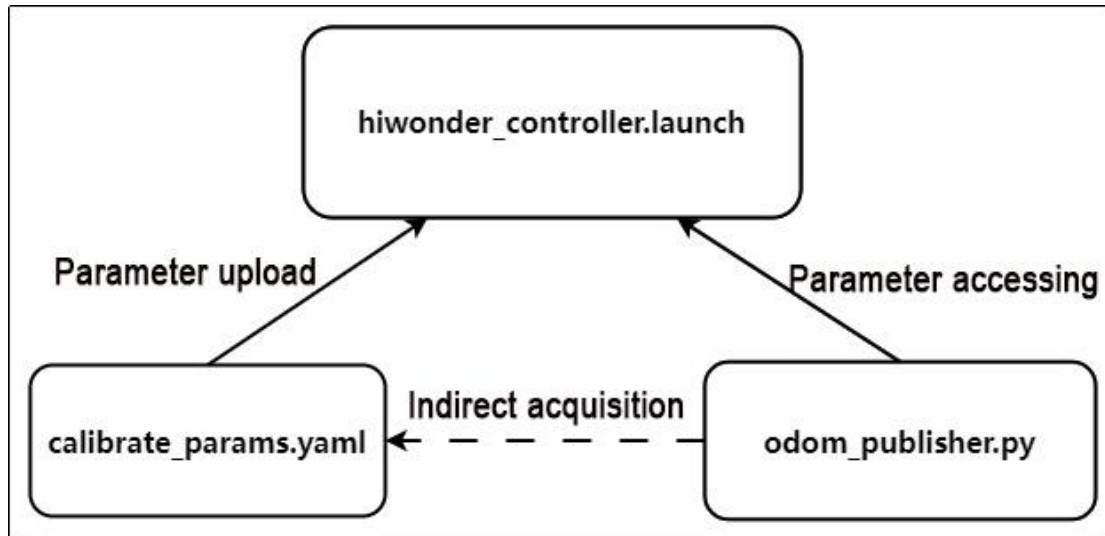
```
sudo systemctl restart start_node.service
```

```

LXTerminal
File Edit Tabs Help
> sudo systemctl restart start_node.service

```

3. Program Analysis

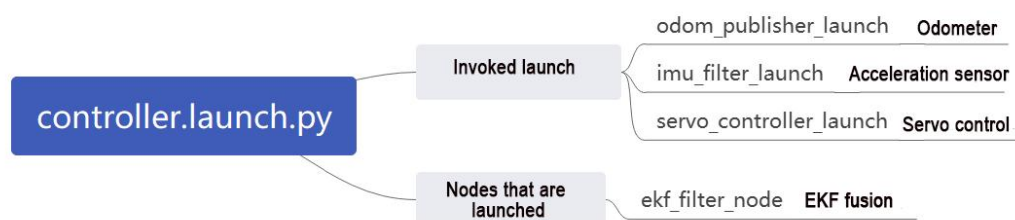


The files include '**controller.launch.py**' for launch configuration, '**calibrate_params.yaml**' for parameter configuration, and '**odom_publisher.py**' for program execution.

During startup, the launch file is executed first. It loads the YAML configuration file and passes the parameters to the ROS nodes. Subsequently, the nodes initialize by reading the configuration parameters from the ROS nodes and communicate with other nodes to collaboratively implement functionalities.

3.1 Launch File

controller.launch.py file description



The launch file is located in:

/home/ubuntu/ros2_ws/src/driver/controller/launch/controller.launch.py


```

1 import os
2 from ament_index_python.packages import get_package_share_directory
3
4 from launch_ros.actions import Node
5 from launch.conditions import IfCondition
6 from nav2_common.launch import ReplaceString
7 from launch import LaunchDescription, LaunchService
8 from launch.substitutions import LaunchConfiguration
9 from launch.launch_description_sources import PythonLaunchDescriptionSource
10 from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription, GroupAction, TimerAction, OpaqueFunction
11
12 def launch_setup(context):
13     compiled = os.environ['need_compile']
14     namespace = LaunchConfiguration('namespace', default='')
15     use_namespace = LaunchConfiguration('use_namespace', default='false').perform(context)
16     use_sim_time = LaunchConfiguration('use_sim_time', default='false')
17     enable_odom = LaunchConfiguration('enable_odom', default='true')
18     map_frame = LaunchConfiguration('map_frame', default='map')
19     odom_frame = LaunchConfiguration('odom_frame', default='odom')
20     base_frame = LaunchConfiguration('base_frame', default='base_footprint')
21     imu_frame = LaunchConfiguration('imu_frame', default='imu_link')
22     frame_prefix = LaunchConfiguration('frame_prefix', default='')
23
24     namespace_arg = DeclareLaunchArgument('namespace', default_value=namespace)
25     use_namespace_arg = DeclareLaunchArgument('use_namespace', default_value=use_namespace)
26     use_sim_time_arg = DeclareLaunchArgument('use_sim_time', default_value=use_sim_time)
27     enable_odom_arg = DeclareLaunchArgument('enable_odom', default_value=enable_odom)
28     map_frame_arg = DeclareLaunchArgument('map_frame', default_value=map_frame)
29     odom_frame_arg = DeclareLaunchArgument('odom_frame', default_value=odom_frame)
30     base_frame_arg = DeclareLaunchArgument('base_frame', default_value=base_frame)
31     imu_frame_arg = DeclareLaunchArgument('imu_frame', default_value=imu_frame)
32     frame_prefix_arg = DeclareLaunchArgument('frame_prefix', default_value=frame_prefix)
33
34     if compiled == 'True':
35         peripherals_package_path = get_package_share_directory('peripherals')
36         controller_package_path = get_package_share_directory('controller')
37     else:
38         peripherals_package_path = '/home/ubuntu/ros2_ws/src/peripherals'
39         controller_package_path = '/home/ubuntu/ros2_ws/src/driver/controller'

```

◆ Set the storage path

Retrieve the paths for the two packages: peripherals and controller.

```

34     if compiled == 'True':
35         peripherals_package_path = get_package_share_directory('peripherals')
36         controller_package_path = get_package_share_directory('controller')
37     else:
38         peripherals_package_path = '/home/ubuntu/ros2_ws/src/peripherals'
39         controller_package_path = '/home/ubuntu/ros2_ws/src/driver/controller'

```

◆ Initiate other Launch files

```

41     odom_publisher_launch = IncludeLaunchDescription(
42         PythonLaunchDescriptionSource([os.path.join(controller_package_path, 'launch/odom_publisher.launch.py')
43     ]),
44     launch_arguments={
45         'namespace': namespace,
46         'use_namespace': use_namespace,
47         'imu_frame': imu_frame,
48         'frame_prefix': frame_prefix,
49         'base_frame': base_frame,
50         'odom_frame': odom_frame
51     }.items()
52 )
53
54     imu_filter_launch = IncludeLaunchDescription(
55         PythonLaunchDescriptionSource([os.path.join(peripherals_package_path, 'launch/imu_filter.launch.py')
56     ]))
57 )

```

odom_publisher_launch Odometer launch

imu_filter_launch IMU launch

◆ Initiate Node

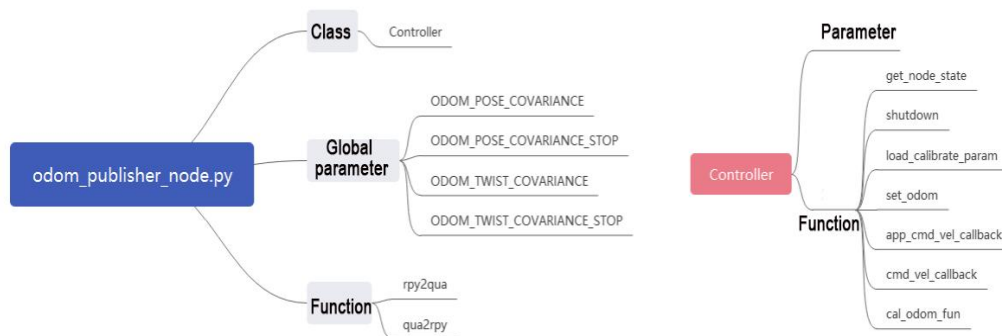
Launch the EKF fusion node.

```

59 if use_namespace == 'false':
60     ekf_param = ReplaceString(source_file=os.path.join(controller_package_path, 'config/ekf.yaml'), replacements={'namespace/': ''})
61 else:
62     ekf_param = ReplaceString(source_file=os.path.join(controller_package_path, 'config/ekf.yaml'), replacements={"namespace/": (namespace, '/')})
63
64 ekf_filter_node = Node(
65     package='robot_localization',
66     executable='ekf_node',
67     name='ekf_filter_node',
68     output='screen',
69     parameters=[ekf_param, {'use_sim_time': use_sim_time}],
70     remappings=[
71         ('/tf', 'tf'),
72         ('/tf_static', 'tf_static'),
73         ('odometry/filtered', 'odom'),
74         ('cmd_vel', 'controller/cmd_vel')
75     ],
76     condition=IfCondition(enable_odom),
77 )

```

3.2 Python Program



The Python program is saved in:

/home/ubuntu/ros2_ws/src/driver/controller/launch/odom_publisher_node.py

◆ Import Library

```

3 import os
4 import math
5 import time
6 import rclpy
7 import signal
8 import tf2_ros
9 import threading
10 from rclpy.node import Node
11 from std_srvs.srv import Trigger
12 from nav_msgs.msg import Odometry
13 from controller import ackermann, mecanum
14 from ros_robot_controller_msgs.msg import MotorsState, BusServoState, SetBusServoState
15 from geometry_msgs.msg import Pose2D, Pose, Twist, PoseWithCovarianceStamped, TransformStamped

```

◆ Main Function

```

275 def main():
276     node = Controller('odom_publisher')
277     rclpy.spin(node) # 循环等待ROS2退出(loop waiting for ROS2 to exit)
278 if __name__ == "__main__":
279     main()

```

The controller class is invoked here, and wait for the node to exit.

◆ Global Parameter


```

17 ODOM_POSE_COVARIANCE = list(map(float,
18     [1e-3, 0, 0, 0, 0, 0,
19     0, 1e-3, 0, 0, 0, 0,
20     0, 0, 1e6, 0, 0, 0,
21     0, 0, 0, 1e6, 0, 0,
22     0, 0, 0, 0, 1e6, 0,
23     0, 0, 0, 0, 0, 1e3]))
24
25 ODOM_POSE_COVARIANCE_STOP = list(map(float,
26     [1e-9, 0, 0, 0, 0, 0,
27     0, 1e-3, 1e-9, 0, 0, 0,
28     0, 0, 1e6, 0, 0, 0,
29     0, 0, 0, 1e6, 0, 0,
30     0, 0, 0, 0, 1e6, 0,
31     0, 0, 0, 0, 0, 1e-9]))
32
33 ODOM_TWIST_COVARIANCE = list(map(float,
34     [1e-3, 0, 0, 0, 0, 0,
35     0, 1e-3, 0, 0, 0, 0,
36     0, 0, 1e6, 0, 0, 0,
37     0, 0, 0, 1e6, 0, 0,
38     0, 0, 0, 0, 1e6, 0,
39     0, 0, 0, 0, 0, 1e3]))
40
41 ODOM_TWIST_COVARIANCE_STOP = list(map(float,
42     [1e-9, 0, 0, 0, 0, 0,
43     0, 1e-3, 1e-9, 0, 0, 0,
44     0, 0, 1e6, 0, 0, 0,
45     0, 0, 0, 1e6, 0, 0,
46     0, 0, 0, 0, 1e6, 0,
47     0, 0, 0, 0, 0, 1e-9]))

```

ODOM_POSE_COVARIANCE Odometer POSE covariance

ODOM_POSE_COVARIANCE_STOP When speed is 0,
Mileage POSE covariance

ODOM_TWIST_COVARIANCE Mileage TWIST covariance

ODOM_TWIST_COVARIANCE_STOP When speed is 0,
Mileage TWIST covariance

◆ Function

```

49 def rpy2qua(roll, pitch, yaw):
50     cy = math.cos(yaw*0.5)
51     sy = math.sin(yaw*0.5)
52     cp = math.cos(pitch*0.5)
53     sp = math.sin(pitch*0.5)
54     cr = math.cos(roll * 0.5)
55     sr = math.sin(roll * 0.5)
56
57     q = Pose()
58     q.orientation.w = cy * cp * cr + sy * sp * sr
59     q.orientation.x = cy * cp * sr - sy * sp * cr
60     q.orientation.y = sy * cp * sr + cy * sp * cr
61     q.orientation.z = sy * cp * cr - cy * sp * sr
62     return q.orientation
63
64 def qua2rpy(x, y, z, w):
65     roll = math.atan2(2 * (w * x + y * z), 1 - 2 * (x * x + y * y))
66     pitch = math.asin(2 * (w * y - x * z))
67     yaw = math.atan2(2 * (w * z + x * y), 1 - 2 * (z * z + y * y))
68
69     return roll, pitch, yaw

```

The function 'rpy2qua' is used to convert Euler angles to quaternions.

The function 'qua2rpy' is used to convert quaternions to Euler angles.

◆ Analysis of the Controller Class

Invoke kinematics

```

87     self.ackermann = ackermann.AckermannChassis(wheelbase=0.145, track_width=0.133, wheel_diameter=0.067)
88     self.mecanum = mecanum.MecanumChassis(wheelbase=0.1368, track_width=0.1446, wheel_diameter=0.065)

```

The 'self.ackermann' calls the Ackermann kinematics and initializes the Ackermann kinematics object.

The 'self.mecanum' calls the Mecanum kinematics and initializes the Mecanum kinematics object.

Define ROS parameters:

```

91     self.declare_parameter('pub_odom_topic', True)
92     self.declare_parameter('base_frame_id', 'base_footprint')
93     self.declare_parameter('odom_frame_id', 'odom')
94     self.declare_parameter('linear_correction_factor', 1.00)
95     self.declare_parameter('angular_correction_factor', 1.00)
96     self.declare_parameter('machine_type', os.environ['MACHINE_TYPE'])
97
98     self.pub_odom_topic = self.get_parameter('pub_odom_topic').value
99     self.base_frame_id = self.get_parameter('base_frame_id').value
100    self.odom_frame_id = self.get_parameter('odom_frame_id').value
101
102    self.linear_factor = self.get_parameter('linear_correction_factor').value
103    self.angular_factor = self.get_parameter('angular_correction_factor').value
104    self.machine_type = self.get_parameter('machine_type').value

```

The function 'self.declare_parameter' is used to define a certain parameter.

The function 'self.get_parameter' is used to obtain a certain parameter.

pub_odom_topic: Whether to publish the odometry node

base_frame_id: Robot footprint ID

odom_frame_id: Robot odometry ID

linear_correction_factor: Linear velocity correction factor

angular_correction_factor: Angular velocity correction factor

machine_type: Type of robot

Publish odometer:

```

107         if self.pub_odom_topic:
108             # self.odom_broadcaster = tf2_ros.TransformBroadcaster(self) #
109             # self.odom_trans = TransformStamped()
110             # self.odom_trans.header.frame_id = self.odom_frame_id
111             # self.odom_trans.child_frame_id = self.base_frame_id
112
113             self.odom = Odometry()
114             self.odom.header.frame_id = self.odom_frame_id
115             self.odom.child_frame_id = self.base_frame_id
116
117             self.odom.pose.covariance = ODOM_POSE_COVARIANCE
118             self.odom.twist.covariance = ODOM_TWIST_COVARIANCE
119
120             self.odom_pub = self.create_publisher(Odometry, 'odom_raw', 1)
121             self.dt = 1.0/50.0
122
123             threading.Thread(target=self.cal_odom_fun, daemon=True).start()

```

Based on the parameter 'pub_odom_topic', determine whether to publish the odometry node. If publishing is required, initialize the node, fill in the corresponding parameters, and publish the odometry using the 'self.create_publisher' function. Update the odometry data using the 'self.cal_odom_fun' function.

Topic Publishing:

```

125         self.motor_pub = self.create_publisher(MotorsState, 'ros_robot_controller/set_motor', 1)
126         self.servo_state_pub = self.create_publisher(SetBusServoState, 'ros_robot_controller/bus_servo/set_state', 1)
127         self.pose_pub = self.create_publisher(PoseWithCovarianceStamped, 'set_pose', 1)
128         self.create_subscription(Pose2D, 'set_odom', self.set_odom, 1)
129         self.create_subscription(Twist, 'controller/cmd_vel', self.cmd_vel_callback, 1)
130         self.create_subscription(Twist, 'cmd_vel', self.app_cmd_vel_callback, 1)
131         self.create_service(Trigger, 'controller/load_calibrate_param', self.load_calibrate_param)
132
133         self.create_service(Trigger, '~/init_finish', self.get_node_state)
134         self.get_logger().info('\033[1;32m%s\033[0m' % 'start')

```

The function 'self.create_subscription' is used to receive topics.

The function 'self.create_service' is used to create services.

self.motor_pub publishes the motor control topic

'ros_robot_controller/set_motor', with the message type MotorsState.

self.servo_state_pub publishes the servo control topic

'ros_robot_controller/bus_servo/set_state', with the message type

SetBusServoState.

self.pose_pub publishes the servo control topic 'set_pose', with the message type PoseWithCovarianceStamped.

Publishes the topic 'set_odom', with the message type Pose2D, and the callback function self.set_odom.

Publishes the topic 'controller/cmd_vel', with the message type Twist, and the callback function self.cmd_vel_callback.

Publishes the topic 'cmd_vel', with the message type Twist, and the callback function self.set_app_cmd_vel_callback.

Publishes the service 'controller/load_calibrate_param', with the service type Trigger, and the callback function self.load_calibrate_param.

Publishes the service '~/init_finish', with the service type Trigger, and the callback function self.get_node_state.

Explanation of Controller Class Functions:

get_node_state	Used to obtain the current node status, as a callback function
shutdown	Used to shut down the ROS node, as a callback function
load_calibrate_param	Used to read the current motion parameters, as a callback function
set_odom	Used to read and set odometry, as a callback function
app_cmd_vel_callback	Used to set the app speed, as a callback function
cmd_vel_callback	Used to set velocity, as a callback function
cal_odom_fun	Used to publish odometry data

3.3 FAQ

Q: The robot continues to move forward even after pressing “Ctrl+C” in the terminal.

A: In such a situation, you need to open a new terminal and enter the command:

rostopic pub/hiwonder_controller/cmd_vel geometry_msgs/Twist “linear:”, then

press Tab to autocomplete. Set the speed to 0, and press Enter to execute.